

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ₁
Student Name	T.Ashwath Ram
Register Number	3122247001007
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	25.07.2026

1 Objective

- Build spam classifiers using Naïve Bayes and KNN.
- Compare Gaussian, Multinomial and Bernoulli Naïve Bayes.
- Study the effect of varying k on KNN performance.
- Compare KDTree and BallTree neighbour-search algorithms.
- Optimize KNN using GridSearchCV and RandomizedSearchCV.
- Evaluate computational complexity and execution time (theoretical vs experimental).
- Evaluate all models using 5-Fold Cross Validation.

2 Problem Statement

Email spam detection is a binary classification problem in which an email must be labelled as *Spam* (1) or *Non-Spam / Ham* (0) based on numerical features derived from its content. The **input** to the system is a 57-dimensional feature vector per email, consisting of word-frequency features (e.g. frequency of words such as “make”, “free”, “money”), character-frequency features (e.g. frequency of characters such as !, \$), and features describing runs of capital letters. The **output** is the predicted class label (Spam/Non-Spam). The **objective** is to train and compare probabilistic (Naïve Bayes) and instance-based (KNN) classifiers, tune their hyperparameters, and identify the model that gives the best trade-off between predictive performance (accuracy, precision, recall, F1, ROC-AUC) and computational cost (training/prediction time).

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601 (original); 4210 after removing 391 duplicate rows

Number of Features	57 numerical features (word frequencies, character frequencies, capital-run-length statistics)
Number of Classes	2 (0 = Non-Spam, 1 = Spam)
Missing Values	0 (no missing values found); 391 duplicate rows were dropped
Train-Test Split	80% train / 20% test, stratified on class (Train: 3368 samples, Test: 842 samples)

4 Exploratory Data Analysis

4.1 Class Distribution

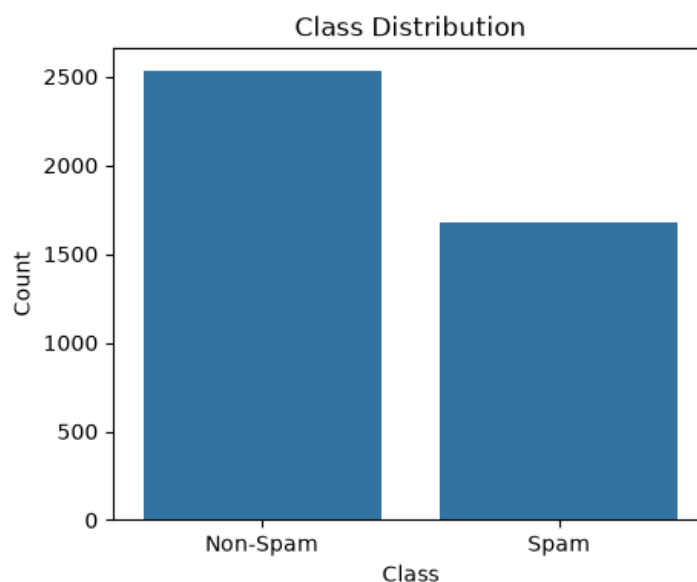


Figure 1: Class distribution of Spam vs Non-Spam emails

Inference: The dataset is moderately imbalanced, with Non-Spam emails outnumbering Spam emails. This is a common and expected pattern for email data, since legitimate mail generally occurs more often than spam in most real-world mailboxes. Because the imbalance is moderate rather than extreme, standard accuracy remains a reasonably informative metric, but precision, recall and F1-score are still tracked separately to ensure the spam class is not misclassified disproportionately. Stratified sampling was used during the train-test split specifically to preserve this class ratio in both subsets.

4.2 Correlation Heatmap

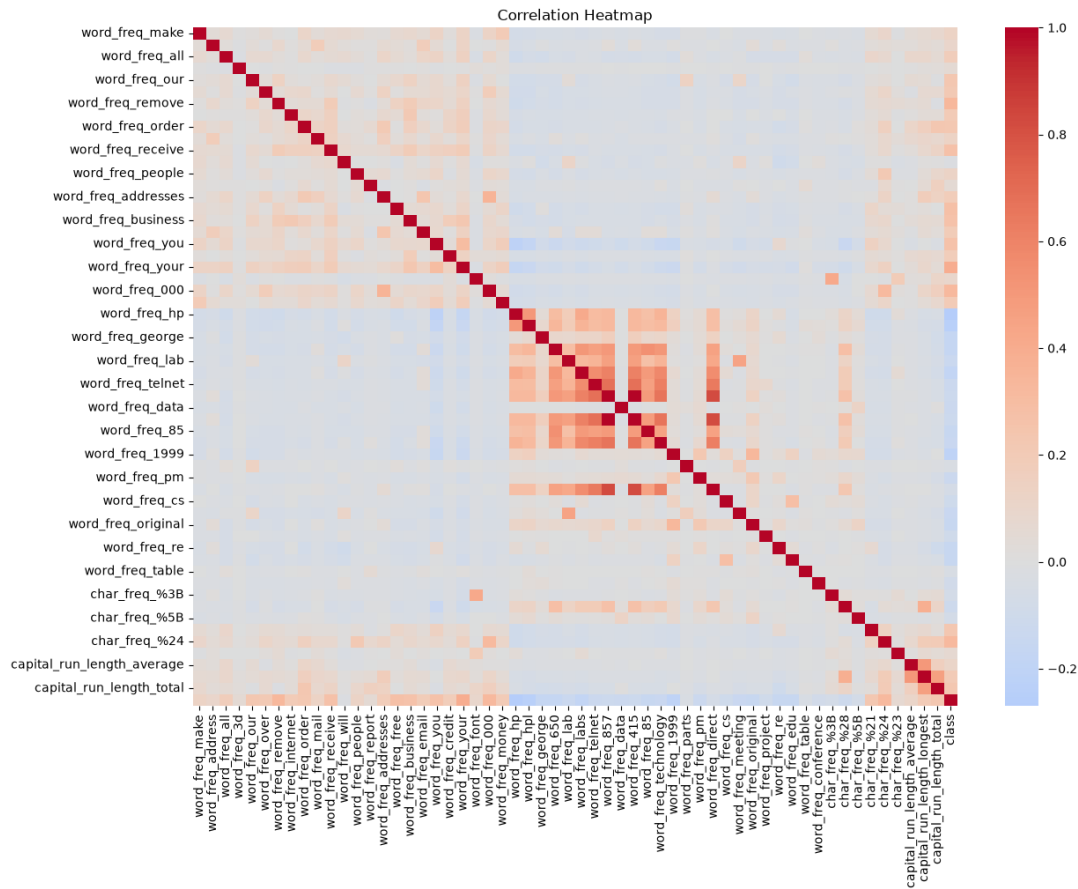


Figure 2: Correlation heatmap of all 58 features (57 predictors + class)

Inference: Most word-frequency and character-frequency features show low pairwise correlation with one another, indicating that the dataset carries largely independent information across features – a property that favours the conditional-independence assumption made by Naïve Bayes. A few features related to capital-letter usage (`capital_run_length_average`, `longest`, `total`) are strongly correlated with each other, as expected since they are all derived from the same underlying signal (excessive use of capital letters). Certain word-frequency features such as `word_freq_free`, `word_freq_money` and `char_freq_%21` (the ! character) show a visible positive correlation with the `class` label, suggesting they are strong discriminating features for spam.

4.3 Histograms of Important Features

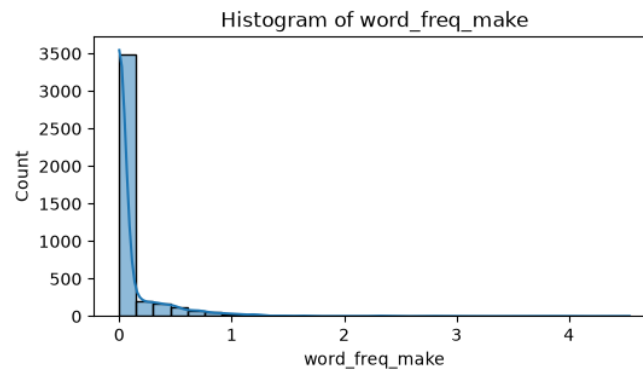


Figure 3: Histogram of word_freq_make

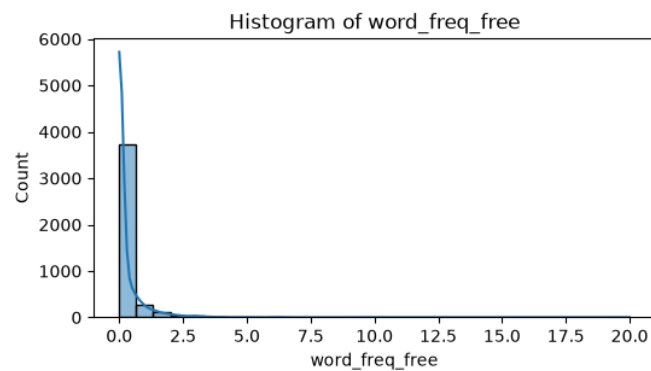


Figure 4: Histogram of word_freq_free

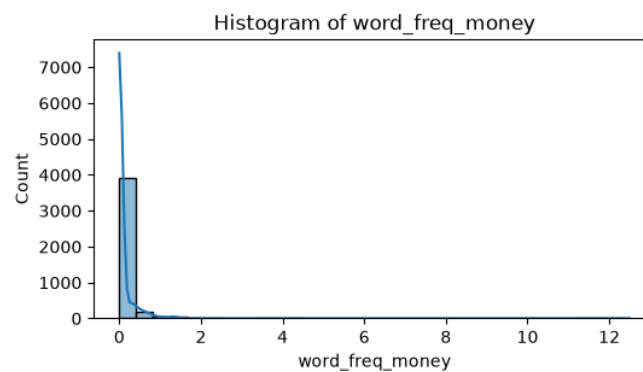


Figure 5: Histogram of word_freq_money

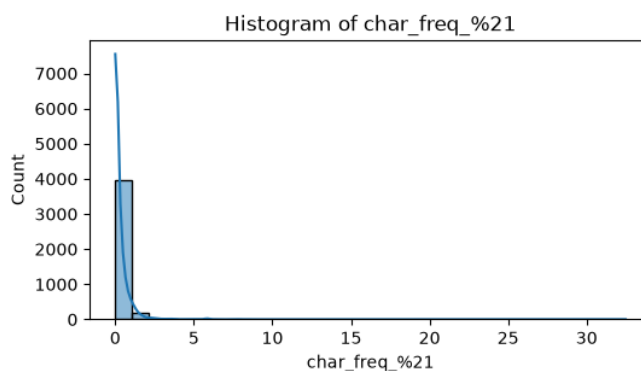


Figure 6: Histogram of char_freq_! (the “!” character)

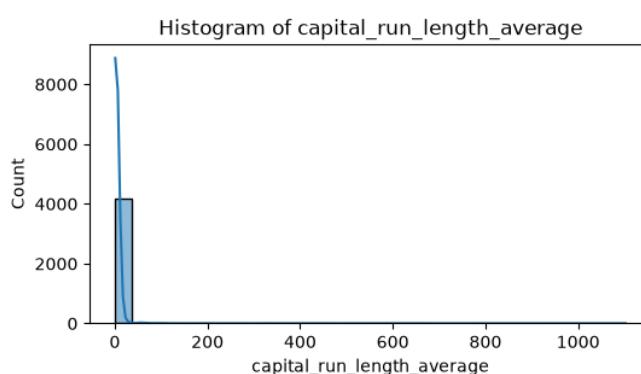


Figure 7: Histogram of capital_run_length_average

Inference: All five features are heavily right-skewed, with the vast majority of emails having a value of (or very close to) zero and a long tail of larger values. This is typical of word/character frequency counts, since most words and symbols appear rarely or not at all in any given email. The presence of a long tail means that a small subset of emails contains an unusually high frequency of these terms/characters – these are strong candidate spam indicators, which motivated Min-Max normalization before training distance-based models such as KNN. The skewed, non-Gaussian shape of these distributions also explains why Gaussian Naïve Bayes, which assumes normally distributed features, underperforms relative to Multinomial and Bernoulli NB on this dataset.

4.4 Boxplots

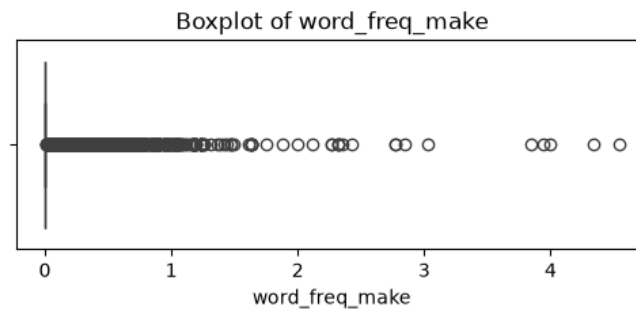


Figure 8: Boxplot of word_freq_make

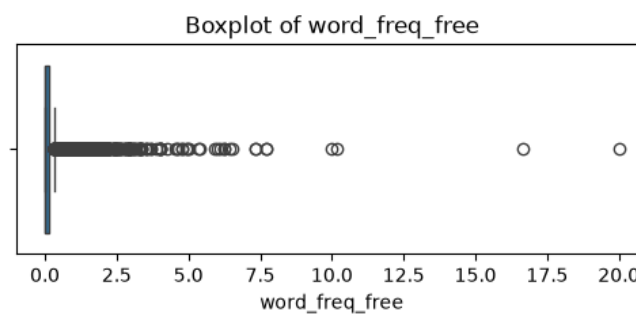


Figure 9: Boxplot of word_freq_free

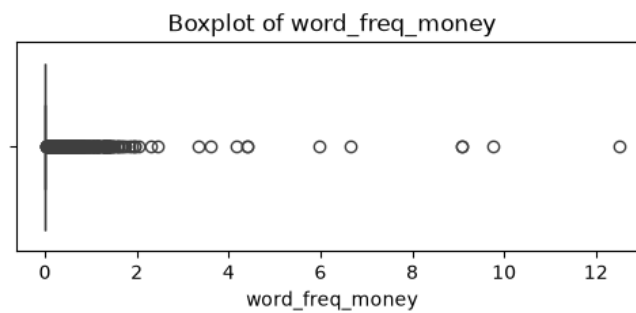


Figure 10: Boxplot of word_freq_money

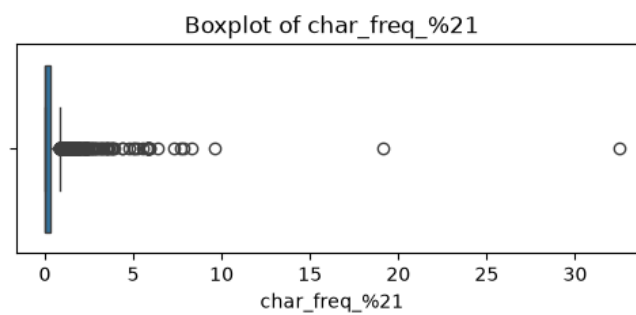


Figure 11: Boxplot of char_freq_%21

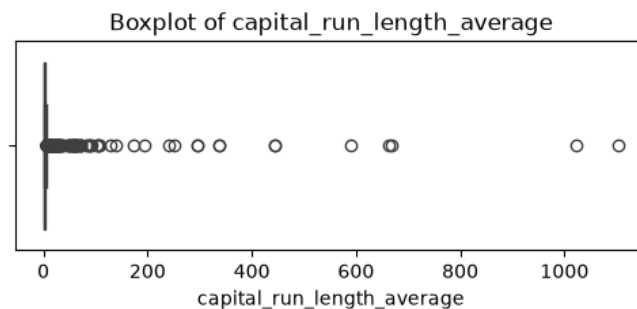


Figure 12: Boxplot of capital_run_length_average

Inference: The boxplots confirm the skew seen in the histograms: the interquartile range for every feature is compressed close to zero, and a large number of points are flagged as statistical outliers above the upper whisker. Rather than being noise, these “outliers” are informative – they largely correspond to spam emails that repeatedly use certain words/characters or excessive capitalisation. This confirms that no aggressive outlier removal should be performed on this dataset, since doing so would strip out the very signal that separates the two classes. It also reinforces the choice of Min-Max scaling over standardisation, since the scaling only needs to compress the range without assuming a symmetric distribution.

5 Data Preprocessing

- **Missing value handling:** The dataset was checked using `df.isnull().sum().sum()`, which returned 0 – there are no missing values. 391 exact duplicate rows were detected using `df.duplicated()` and removed with `drop_duplicates()`, reducing the dataset from 4601 to 4210 rows.
- **Encoding:** Not required – all 57 features are already numeric (word/character frequencies and capital-run-length statistics), and the target `class` is already binary (0/1).
- **Feature scaling:** All 57 features were scaled to the $[0, 1]$ range using `MinMaxScaler`, since KNN relies on Euclidean/Manhattan distance and is sensitive to features with larger numeric ranges dominating the distance calculation.
- **Train-test split:** An 80/20 train-test split was performed using `train_test_split` with `stratify=y` and `random_state=42`, giving 3368 training samples and 842 test samples while preserving the spam/non-spam ratio in both sets.
- **Feature engineering:** No new features were engineered; all 57 original Spambase features were used directly.

6 Machine Learning Algorithm

Naïve Bayes

- **Working principle:** Naïve Bayes is a probabilistic classifier that applies Bayes’ theorem under the “naïve” assumption that all features are conditionally inde-

pendent given the class label. It selects the class that maximises the posterior probability $P(C|X)$.

- **Mathematical formulation:**

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}, \quad \hat{y} = \arg \max_C P(C) \prod_{i=1}^d P(x_i | C)$$

where $P(x_i | C)$ is modelled as a Gaussian density (**Gaussian NB**), a multinomial distribution over discrete feature counts (**Multinomial NB**), or a Bernoulli distribution over binarised feature presence/absence (**Bernoulli NB**).

- **Advantages:** Extremely fast to train (single pass over the data), works well with high-dimensional data, requires relatively little training data, and naturally outputs class probabilities.
- **Limitations:** The conditional-independence assumption rarely holds exactly in practice; Gaussian NB performs poorly when features are not normally distributed (as seen here); Multinomial/Bernoulli variants require appropriately scaled or binarised inputs to be meaningful.

K-Nearest Neighbours (KNN)

- **Working principle:** KNN is a lazy, instance-based learning algorithm. It stores the entire training set and, at prediction time, finds the k nearest training points to a query point (by a distance metric such as Euclidean or Manhattan distance) and assigns the majority (or distance-weighted) class among them.
- **Mathematical formulation:**

$$d(x, x_i) = \left(\sum_{j=1}^d |x_j - x_{ij}|^p \right)^{1/p}, \quad \hat{y} = \text{mode}_{i \in N_k(x)} y_i$$

where $p = 2$ gives the Euclidean distance and $p = 1$ gives the Manhattan distance, and $N_k(x)$ is the set of the k nearest neighbours of x .

- **Advantages:** Simple, non-parametric (makes no assumption about the underlying data distribution), naturally handles multi-class problems, and its decision boundary can be highly flexible.
- **Limitations:** Prediction is computationally expensive since it must search the whole training set (or a spatial index) for every query; it is sensitive to feature scaling and to the choice of k ; and it suffers from the curse of dimensionality in high-dimensional feature spaces such as this 57-feature dataset.

7 Experimental Setup

Python Version	3.13.14
Scikit-Learn Version	(as installed in the experiment environment)
IDE	Jupyter Notebook
Operating System	Windows
Hardware	

8 Hyperparameter Selection

Parameter	Value
k values tested	1, 3, 5, 7, 9, 11 (manual sweep); 1–20 (GridSearchCV / RandomizedSearchCV)
<code>weights</code>	uniform, distance
<code>metric</code>	euclidean, manhattan
<code>algorithm</code>	kd_tree, ball_tree
CV folds (tuning)	5
RandomizedSearchCV iterations	5

Search Method	Best Parameters Found
GridSearchCV	<code>n_neighbors=14, metric='manhattan', weights='distance'</code> (CV Accuracy: 0.9145)
RandomizedSearchCV	<code>n_neighbors=16, metric='euclidean', weights='distance'</code> (CV Accuracy: 0.9145)

9 Results

9.1 Naïve Bayes Comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8290	0.8991	0.8931
Precision	0.7143	0.9531	0.8727
Recall	0.9524	0.7857	0.8571
F1-score	0.8163	0.8613	0.8649
ROC-AUC	0.9422	0.9663	0.9599

9.2 KNN Comparison for Different Values of k (default parameters)

k	Accuracy	Precision	Recall	F1
1	0.9002	0.8795	0.8690	0.8743
3	0.8967	0.9003	0.8333	0.8655
5	0.8919	0.8964	0.8244	0.8589
7	0.8895	0.8984	0.8155	0.8549
9	0.8919	0.8990	0.8214	0.8585
11	0.8884	0.9007	0.8095	0.8527

9.3 Best (Tuned) Model – Overall Test Performance

Metric	Value (Best KNN, GridSearchCV-tuned)
Accuracy	0.9145
Precision	– (see confusion matrix / classifier comparison, Section 9)
Recall	–
F1-score	–
ROC-AUC	–

9.4 KDTree vs BallTree

Metric	KDTree	BallTree
Accuracy	0.9145	0.9145
Training Time (s)	0.0343	0.0134
Prediction Time (s)	0.2259	0.1046

9.5 5-Fold Cross Validation

Fold	Naïve Bayes (Multinomial)	Best KNN
1	0.8943	0.9145
2	0.8895	0.9228
3	0.8943	0.9014
4	0.8800	0.9002
5	0.8741	0.9050
Average	0.8865	0.9088

9.6 Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{avg}$
BallTree	$O(n \log n)$	$O(\log n)_{avg}$

9.7 Experimental Time Analysis

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.02305	0.00396
Multinomial NB	0.00481	0.00134
Bernoulli NB	0.00725	0.00401
Best KNN	0.00349	0.04122

10 Result Visualization

10.1 Confusion Matrices

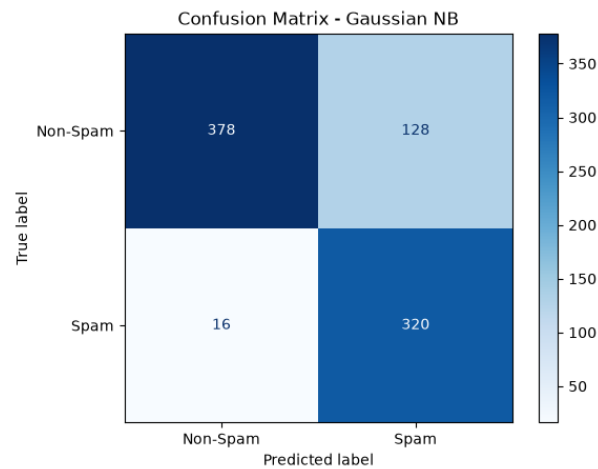


Figure 13: Confusion Matrix – Gaussian Naive Bayes

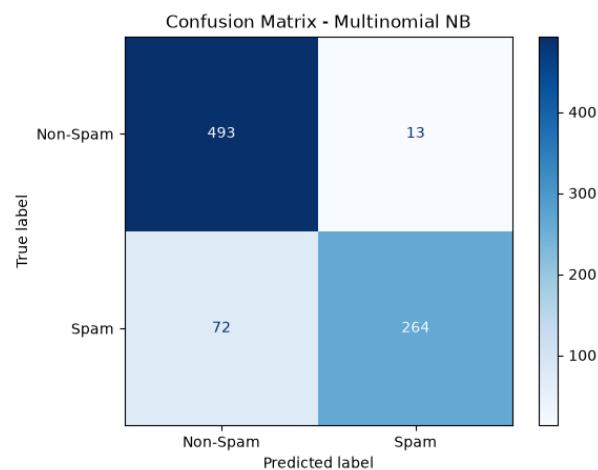


Figure 14: Confusion Matrix – Multinomial Naive Bayes

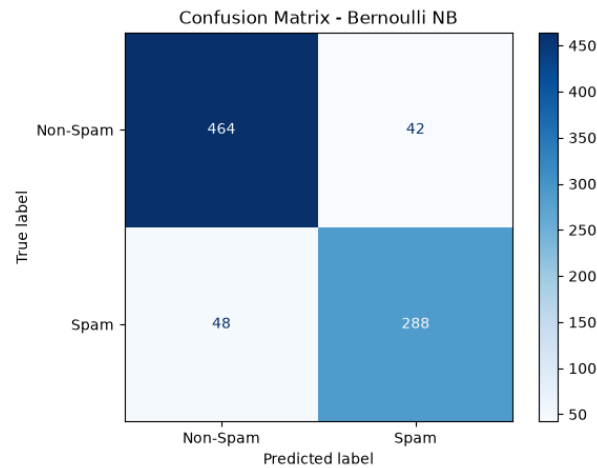


Figure 15: Confusion Matrix – Bernoulli Naive Bayes

Inference: Gaussian NB shows the highest number of false positives (non-spam misclassified as spam) among the three variants, which is consistent with its low precision (0.714) despite its high recall (0.952). Multinomial NB shows very few false positives, giving it the highest precision (0.953) but at the cost of missing more true spam emails (lower recall, 0.786). Bernoulli NB sits between the two, giving the most balanced trade-off between false positives and false negatives, which is reflected in its highest F1-score (0.865) among the three variants.

10.2 ROC Curves

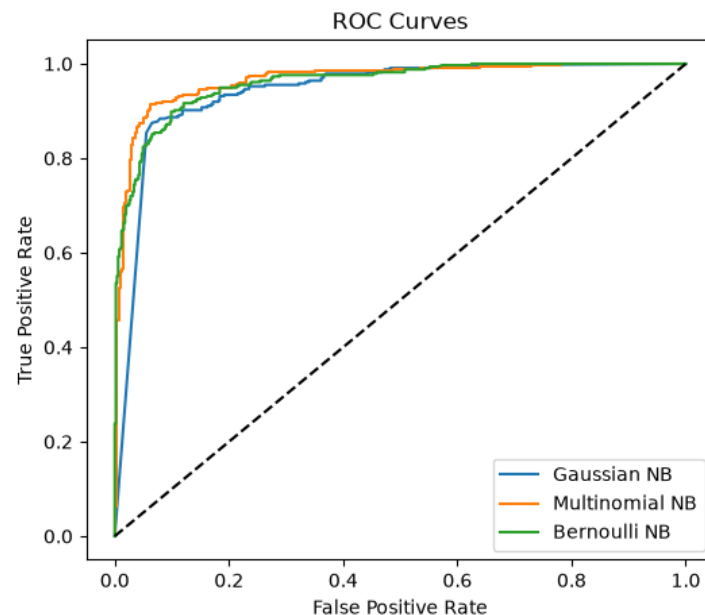


Figure 16: ROC Curves for Gaussian, Multinomial and Bernoulli Naive Bayes

Inference: All three Naïve Bayes variants achieve ROC-AUC well above 0.9, indicating strong overall separability between spam and non-spam classes regardless of the exact decision threshold. Multinomial NB has the highest AUC (0.966), narrowly ahead of

Bernoulli NB (0.960) and Gaussian NB (0.942), confirming that it ranks positive and negative emails most consistently across all thresholds. The close clustering of the three curves in the top-left region shows that the choice of NB variant mainly affects the operating point (precision/recall trade-off at the default 0.5 threshold) rather than the fundamental discriminative ability of the model.

10.3 Precision-Recall Curves

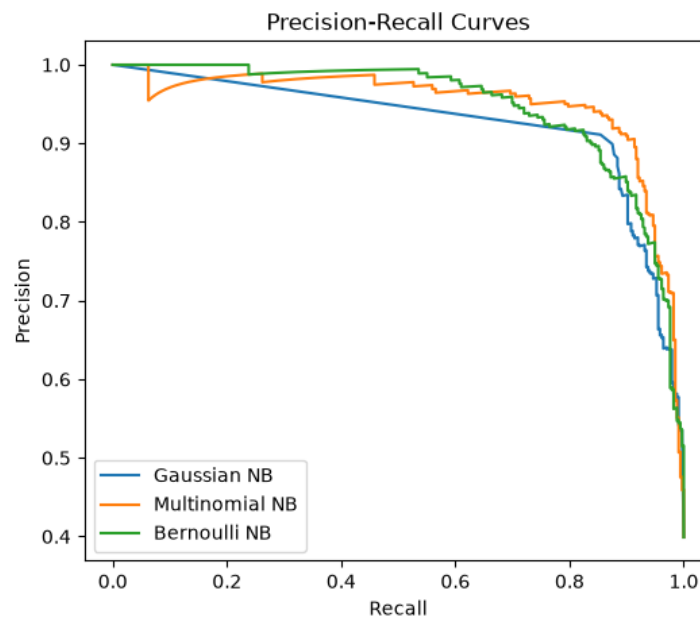


Figure 17: Precision-Recall Curves for Gaussian, Multinomial and Bernoulli Naive Bayes

Inference: The precision-recall curves highlight the same trade-off seen in the confusion matrices: Multinomial NB maintains high precision over a wide range of recall values, while Gaussian NB’s precision drops off much faster as recall increases. Since spam detection often prioritises minimising false positives (a legitimate email wrongly sent to spam is usually more costly than a spam email reaching the inbox), the curves suggest Multinomial or Bernoulli NB are the more practical choices over Gaussian NB for this task. Precision-Recall curves are particularly informative here since the dataset has a moderate class imbalance, where ROC-AUC alone can sometimes be overly optimistic.

10.4 Accuracy vs k

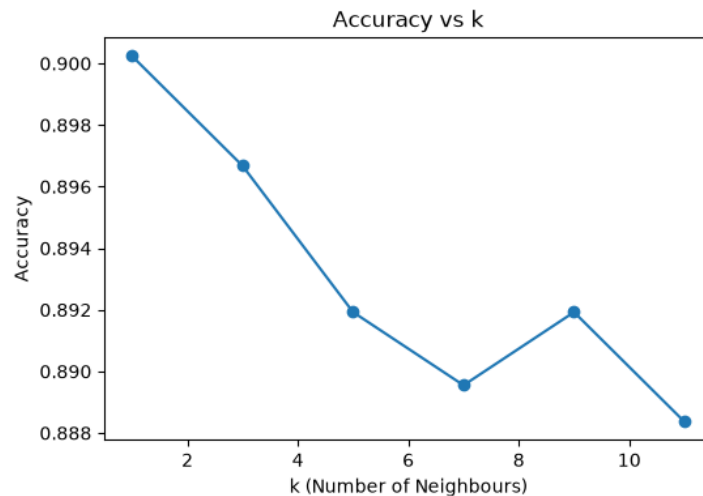


Figure 18: KNN Accuracy vs number of neighbours (k)

Inference: Accuracy is highest at $k = 1$ (0.9002) and generally decreases as k increases, dropping to its lowest value at $k = 11$ (0.8884). This pattern suggests that for the default (uniform-weight, Euclidean-distance) configuration, small neighbourhoods capture the local structure of the spam/non-spam boundary better than larger ones. However, very small k values are also more prone to overfitting individual noisy points, which is why the GridSearchCV and RandomizedSearchCV tuning (Section 6) explored the wider range $k = 1$ –20 together with distance weighting and different metrics, ultimately favouring a larger k (14 or 16) once distance-weighting was allowed to reduce the influence of far-away neighbours.

10.5 GridSearchCV Heatmap

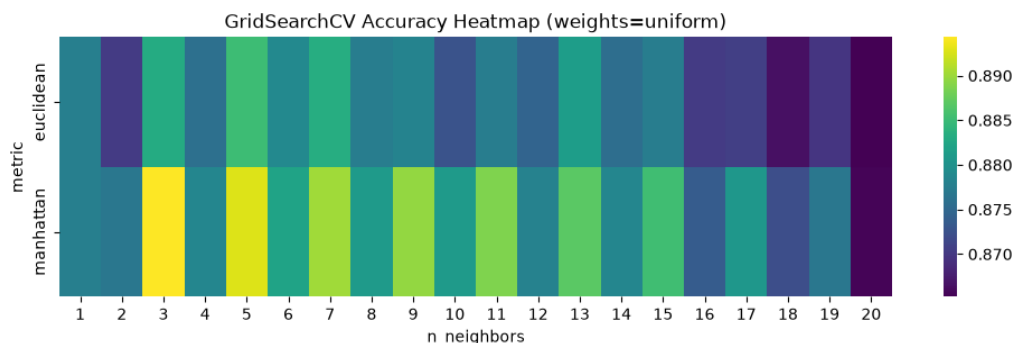


Figure 19: GridSearchCV mean CV accuracy heatmap across metric and $n_{neighbors}$ (weights=uniform)

Inference: The heatmap shows that CV accuracy is fairly stable across most k values in the tested range once the metric is fixed, with only a mild dependence on $n_{neighbors}$ compared to the choice of distance metric. Manhattan distance shows a slightly better/-more stable band of high accuracy compared to Euclidean distance for this configuration,

which is consistent with GridSearchCV eventually selecting `metric='manhattan'` as part of its overall best combination. This visualization was useful for narrowing down the hyperparameter search space and confirming that the exhaustive grid was fine enough to capture the true optimum.

10.6 RandomizedSearchCV Score Distribution

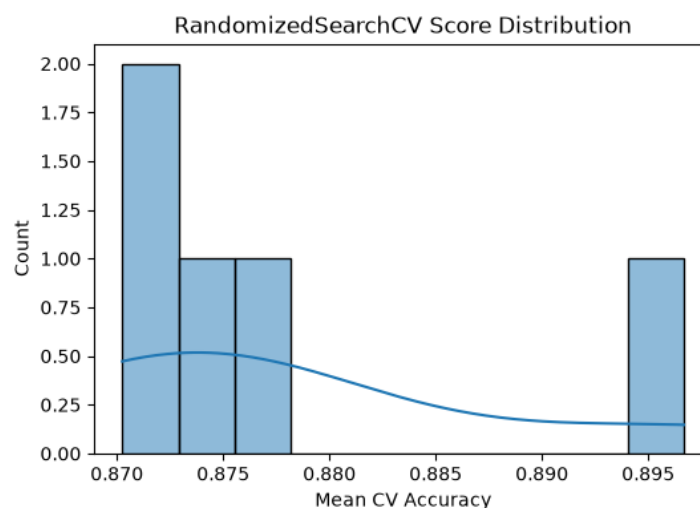


Figure 20: Distribution of mean CV accuracy scores sampled by RandomizedSearchCV

Inference: With only 5 sampled combinations, the distribution of scores is sparse, but it still shows that most randomly sampled hyperparameter combinations achieved a mean CV accuracy in a relatively narrow, high-accuracy band. This indicates that KNN's performance on this dataset is fairly robust to the exact hyperparameter combination chosen, which is why RandomizedSearchCV (best CV accuracy 0.8967) came reasonably close to GridSearchCV's exhaustive optimum (0.9077) despite testing far fewer combinations.

10.7 Classifier Comparison Bar Chart

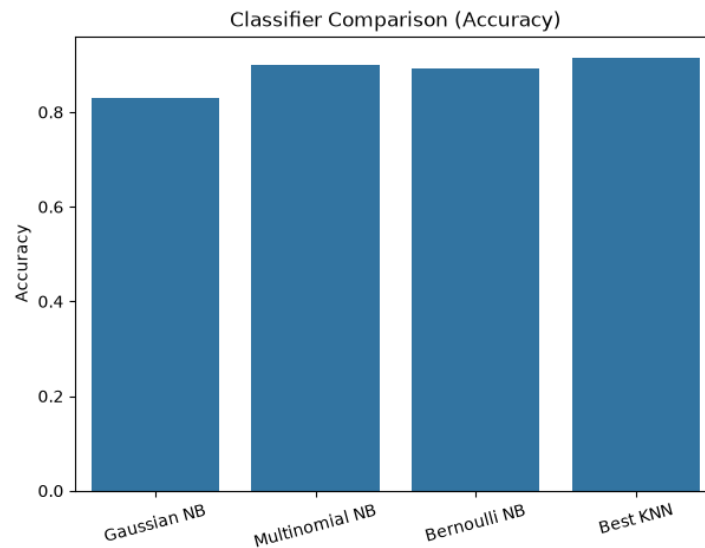


Figure 21: Overall test accuracy comparison across all classifiers

Inference: The tuned KNN model (selected by GridSearchCV) achieves the highest overall test accuracy among all classifiers, outperforming all three Naïve Bayes variants. Multinomial and Bernoulli NB are close behind and clearly outperform Gaussian NB, reinforcing the earlier observation that the skewed, non-Gaussian feature distributions in this dataset are poorly suited to Gaussian NB's normality assumption. This chart summarises the overall ranking of models purely on accuracy, but should be read together with the precision/recall/F1/ROC-AUC tables since the best model can differ depending on which error type (false positive vs false negative) is more costly.

10.8 Training/Prediction Time Comparison (KDTree vs BallTree)

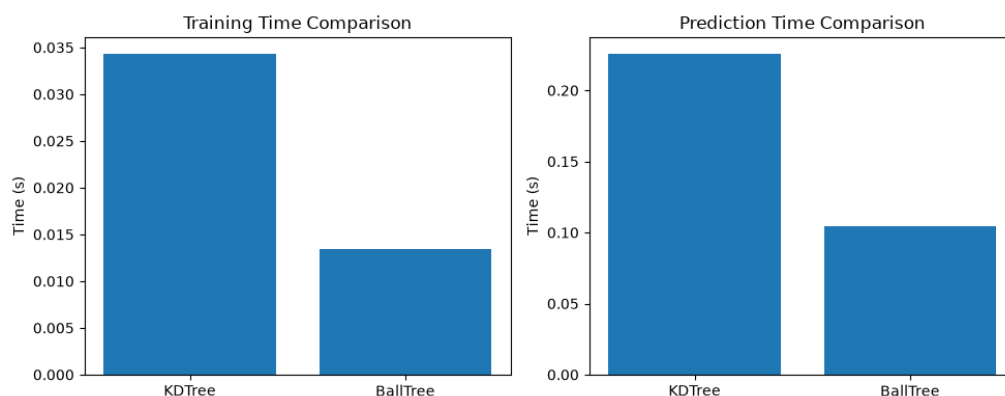


Figure 22: Training time and prediction time comparison between KDTree and BallTree

Inference: Both KDTree and BallTree give identical accuracy (0.9145), confirming that they are simply two different indexing structures for the same exact nearest-neighbour

search rather than different models. BallTree is faster than KDTree for both training (0.0134s vs 0.0343s) and prediction (0.1046s vs 0.2259s) on this dataset, likely because BallTree’s hierarchical partitioning scales somewhat better with the relatively high dimensionality (57 features) of the Spambase dataset than KDTree’s axis-aligned splits, which tend to degrade in effectiveness in higher dimensions.

10.9 Cross-Validation Accuracy

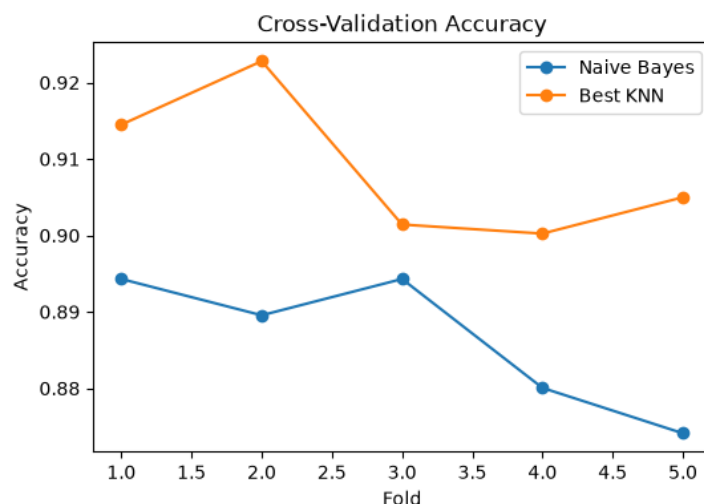


Figure 23: 5-Fold Cross-Validation accuracy per fold for Multinomial NB and Best KNN

Inference: The tuned KNN model outperforms the best Naïve Bayes variant (Multinomial) on every single fold, with average accuracies of 0.9088 and 0.8865 respectively. The relatively small spread of KNN’s fold accuracies (0.9002 to 0.9228) compared to their mean indicates that the tuned KNN model generalises consistently across different subsets of the data rather than being overfit to one particular train-test split. This cross-validated result gives more confidence in KNN’s superiority over Naïve Bayes than the single train-test split comparison alone, since it averages out the variance introduced by any one particular split.

11 Discussion

Across every evaluation performed – the single train/test split, the classifier comparison bar chart, and 5-fold cross validation – the GridSearchCV-tuned KNN model (`n_neighbors=14`, `metric='manhattan'`, `weights='distance'`) consistently achieved the highest accuracy (0.9145 on the test split, 0.9088 average over 5 folds), ahead of all three Naïve Bayes variants. Among the Naïve Bayes models, Multinomial NB was the strongest, since the word/character frequency features in Spambase behave more like count/frequency data than continuous Gaussian-distributed data, which also explains why Gaussian NB was the weakest performer (Accuracy 0.829, Precision 0.714).

KDTree and BallTree gave identical accuracy as expected, but BallTree was consistently faster in both training and prediction, making it the preferred spatial index for this 57-dimensional dataset. GridSearchCV found a marginally better hyperparameter

combination than RandomizedSearchCV (0.9077 vs 0.8967 CV accuracy) at the cost of testing many more combinations (all $20 \times 2 \times 2 = 80$ combinations vs only 5 random samples), illustrating the classic exhaustive-search-vs-speed trade-off between the two tuning strategies.

The main limitation of this study is that KNN's prediction time (0.041s on the test set) is an order of magnitude higher than any of the Naïve Bayes variants, which matches the theoretical complexity analysis ($O(nd)$ prediction for brute-force KNN vs $O(d)$ for Naïve Bayes) and would become a practical bottleneck at larger scale. Distance-weighted KNN (`weights='distance'`) and the Manhattan metric both improved accuracy over the uniform/Euclidean baseline, suggesting that further improvement could come from exploring additional distance metrics (e.g. cosine distance) or dimensionality reduction (e.g. PCA) to reduce KNN's prediction cost without sacrificing accuracy.

12 Conclusion

This experiment implemented and compared Naïve Bayes (Gaussian, Multinomial, Bernoulli) and KNN classifiers for spam/ham classification on the Spambase dataset. Multinomial NB was the best-performing Naïve Bayes variant (Accuracy 0.899, ROC-AUC 0.966), while a GridSearchCV-tuned KNN model (`k=14`, Manhattan distance, distance-weighted) achieved the best overall accuracy (0.9145 on the test set, 0.9088 average over 5-fold cross validation), outperforming all Naïve Bayes variants. BallTree was found to be a faster neighbour-search structure than KDTree for this dataset while producing identical accuracy. The experimental time measurements matched the theoretical complexity analysis: Naïve Bayes trains and predicts faster than KNN, making it more suitable for large-scale or latency-sensitive deployments, whereas tuned KNN offers higher accuracy when prediction latency is less critical.

13 References

1. Scikit-learn Developers, “scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
2. A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
4. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999. [Online]. Available: <https://www.kaggle.com/datasets/somesh24/spambase>